1 General Discussion

The architecture for this MNIST accelerator relies upon a parametrizable PE count. The network will have multiple layers, with 784 activations in the input layer and 10 activations in the output layer. The PE count, due to the parametrizability requirement, will not be tied to the number of output neurons.

Each process engine computes one output neuron at a time, meaning each layer may therefore require several PE passes / tiles. For the purposes of this explanation, a tile refers to the chunk of a layer that is computed in a single PE pass. The way each process engine computes the activation for its corresponding output neuron is more easily explained if the matrix multiplication is shown explicitly:

$$\begin{bmatrix} a_0^1 \\ a_1^1 \\ \vdots \\ a_{m-1}^1 \end{bmatrix} = \text{RELU}\left(\begin{bmatrix} w_{0,0}^0 & w_{0,1}^0 & \cdots & w_{0,n-1}^0 \\ w_{1,0}^0 & w_{1,1}^0 & \cdots & w_{1,n-1}^0 \\ \vdots & \vdots & \ddots & \vdots \\ w_{m-1,0}^0 & w_{m-1,1}^0 & \cdots & w_{m-1,n-1}^0 \end{bmatrix} \cdot \begin{bmatrix} a_0^0 \\ a_1^0 \\ \vdots \\ a_{n-1}^0 \end{bmatrix} + \begin{bmatrix} b_0^1 \\ b_1^1 \\ \vdots \\ b_{m-1}^1 \end{bmatrix} \right) \tag{1}$$

In the equation above, the vector formed by the activations a_i^1 are the activations computed in the next layer. In general, the activation a_i^j represents the activation value stored in neuron i on layer j. The weight matrix contains weights $w_{i,j}^k$, where i is the index of the output neuron associated with the weight, j is the index of the input neuron associated with the weight, and k is the layer the input neuron belongs to. The diagram below shows the relationship between the activations and weights; however, it should be noted that there are many more neurons and, consequently, weights in all layers. Three indices for the weights and two for the neurons is required for there to be no ambiguity as to how neurons affect their neighbors.

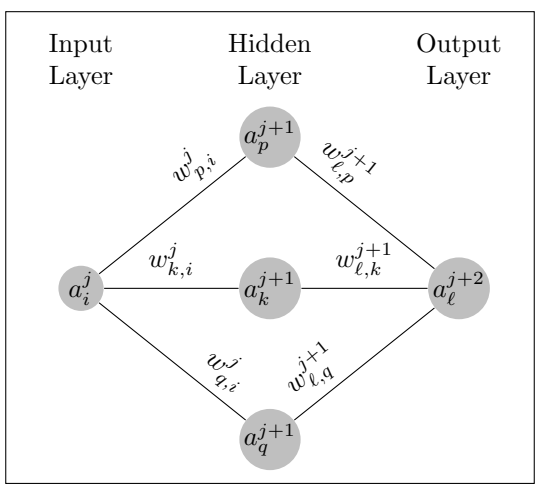

Figure 1: Structure of the Neural Network layers j through $j+2$.

Another thing that will help out with seeing how data is streamed to the process engines is if I

expand equation 1:

$$
\begin{aligned}
a_0^1 &= \text{RELU}\left(w_{0,0}^0 a_0^0 + w_{0,1}^0 a_1^0 + \cdots + w_{0,n-1}^0 a_{n-1}^0 + b_0^1\right) \\
a_1^1 &= \text{RELU}\left(w_{1,0}^0 a_0^0 + w_{1,1}^0 a_1^0 + \cdots + w_{1,n-1}^0 a_{n-1}^0 + b_1^1\right) \\
&\vdots \\
a_i^1 &= \text{RELU}\left(\sum_{j=0}^{n-1} w_{i,j}^0 a_j^0 + b_i^1\right) \\
&\vdots \\
a_{m-1}^1 &= \text{RELU}\left(w_{m-1,0}^0 a_0^0 + w_{m-1,1}^0 a_1^0 + \cdots + w_{m-1,n-1}^0 a_{n-1}^0 + b_{m-1}^1\right)
\end{aligned}
$$

Notice that the first term in each dot product in the expanded matrix multiplication contains the same activation a_0^0, but a different weight. From this information, it is easy to see that when we setup the control unit to stream data to the process engines, each process engine should receive the *same* activation, but a different weight on a given cycle. After MAC operation terminates, each PE will then add a bias and then apply RELU, and after doing so, each PE will have computed a different activation. For example, suppose we have two process engines. In the first cycle, PE 1 will compute a_0^1 and PE 2 will compute a_1^1. These two activations a_0^1 and a_1^1 are contained within a single tile. You can see how by increasing the number of process engines, the more activations belong in a single tile, and the fewer tiles are needed to compute all activations in a single layer. Concretely, this relationship is given by:

$$
\text{Number of Output Tiles} = \left\lceil \frac{N_{out}}{\text{NUM_PE}} \right\rceil, \tag{2}
$$

where N_{out} are the number of neurons in the next layer. The control unit will first split the N_{out} neurons into tiles of size NUM_PE. Then, for each tile, it will clear all active accumulators, stream N_{in} activations over time, then each PE multiplies a common activation by its own weight for its assigned output neuron, then after N_{in} MAC cycles, apply that PE's bias, then apply activation, then store the tile's results. Then repeat this until all output neurons in the layer are finished. The pseudocode below shows the general idea for this procedure.

Algorithm 1 Control Unit MAC Procedure

```
for each output tile t: do
    clear accumulators
    for k in 0 ... N_in − 1: do
        read activation[k]
        read weights for outputs in tile t at input index k
        MAC on all active PEs
    end for
    apply biases for tile t
    apply activation for tile t
    store results for tile t
end for
move on to next layer
```

Because NUM_PE is parametrizable, we must be able to handle the special case where $N_{out} \not\equiv 0$ mod NUM_PE, which would be the case if, for example, $N_{out} = 10$ and NUM_PE $= 4$. In this scenario, there will be three tiles, where tile 0 contains outputs 0,1,2, and 3; tile 1 contains outputs

4, 5, 6, and 7; and tile 3 contains outputs 8 and 9, plus two unused PEs. To handle this case, an active mask will be used where we may have something like

```
      TILE 1              TILE 2              TILE 3
pe_active[0] = 1; | pe_active[0] = 1; | pe_active[0] = 1;
pe_active[1] = 1; | pe_active[1] = 1; | pe_active[1] = 1;
pe_active[2] = 1; | pe_active[2] = 1; | pe_active[2] = 0;
pe_active[3] = 1; | pe_active[3] = 1; | pe_active[3] = 0;
```

applied to respective tiles.

To implement all of these ideas, the PE state machine should have the states CLEAR, READ/MAC (repeated over all N_{in}), BIAS, ACT, STORE, and ADVANCE_TILE or ADVANCE_LAYER (depending on which tile we are on).

2 Memory Unit

The plan was initially to separate the memory unit into activation memory and weight memory. This idea still makes sense, but each memory block will be handled differently. Activation memory is simple; the control unit will fetch one activation per cycle for the current layer input vector, meaning that the activation memory may be split into vectors, one for each layer. Then if we want to compute the activations in layer i, we start by fetching the first activation in the vector $i-1$ (streaming it to all PEs), then on the next cycle fetch the next activation, and so on. Then the process engines will write to the currently empty vector i.

The weight memory will be more complex; the control unit needs to fetch NUM_PE weights per cycle for the current tile and current input index. This may be achieved with, for example, NUM_PE weight banks. The idea is that we should be able to fetch a weight vector $[w_0(k), w_1(k), ..., w_{\text{NUM_PE}-1}(k)]$, where each $w_i(k)$ belongs to the output neuron assigned to PE i in the current tile. Figure 2 depicts this idea more clearly.

Figure 2 shows an architecture with 4 PEs, 5 neurons in hidden layer 1, and 6 in hidden layer 2. Input activations are streamed serially to all PEs, while the weight memory is partitioned into four banks, one per PE, each read serially. At each cycle, every PE receives the same activation and a distinct weight from its corresponding bank. This continues for n cycles to compute four hidden layer activations (a_0^1–a_3^1), which are written in order for later serial access.

Since hidden layer 1 has 5 neurons, a second pass computes a_4^1 using only one active PE while the others remain idle. The process then repeats for hidden layer 2, taking m cycles per pass due to shorter input vectors. Here, $n = 784$ (input size) and $m = 5$ (hidden layer 1 size). The weight memory contains corresponding vectors for each layer (6 for hidden layer 2), with layer divisions shown only for clarity; an additional set for the 10-neuron output layer is omitted from the figure.

To be more concrete, because each PE serves distinct neurons, so do the weight memory banks. In our example, bank 0 (the bank values in the image should be reindexed from 0) stores neuron 0 weights (784 entries) and neuron 4 weights (784 entries). Bank 1 stores neuron 1 weights, bank 2 stores neuron 2 weights, and bank 3 stores neuron 3 weights. In other words:

```
     TILE 1                TILE 2
BANK 0 -> NEURON 0 | BANK 0 -> NEURON 4
BANK 1 -> NEURON 1 | BANK 1 -> UNUSED
BANK 2 -> NEURON 2 | BANK 2 -> UNUSED
BANK 2 -> NEURON 3 | BANK 3 -> UNUSED
```

Thus, during tile t, bank p should output weights for $n = t * NUM_PE + p$. Additionally, each bank only needs to use the same address: $addr = t * N_{in} + in_idx$, where in_idx is incremented on each cycle.

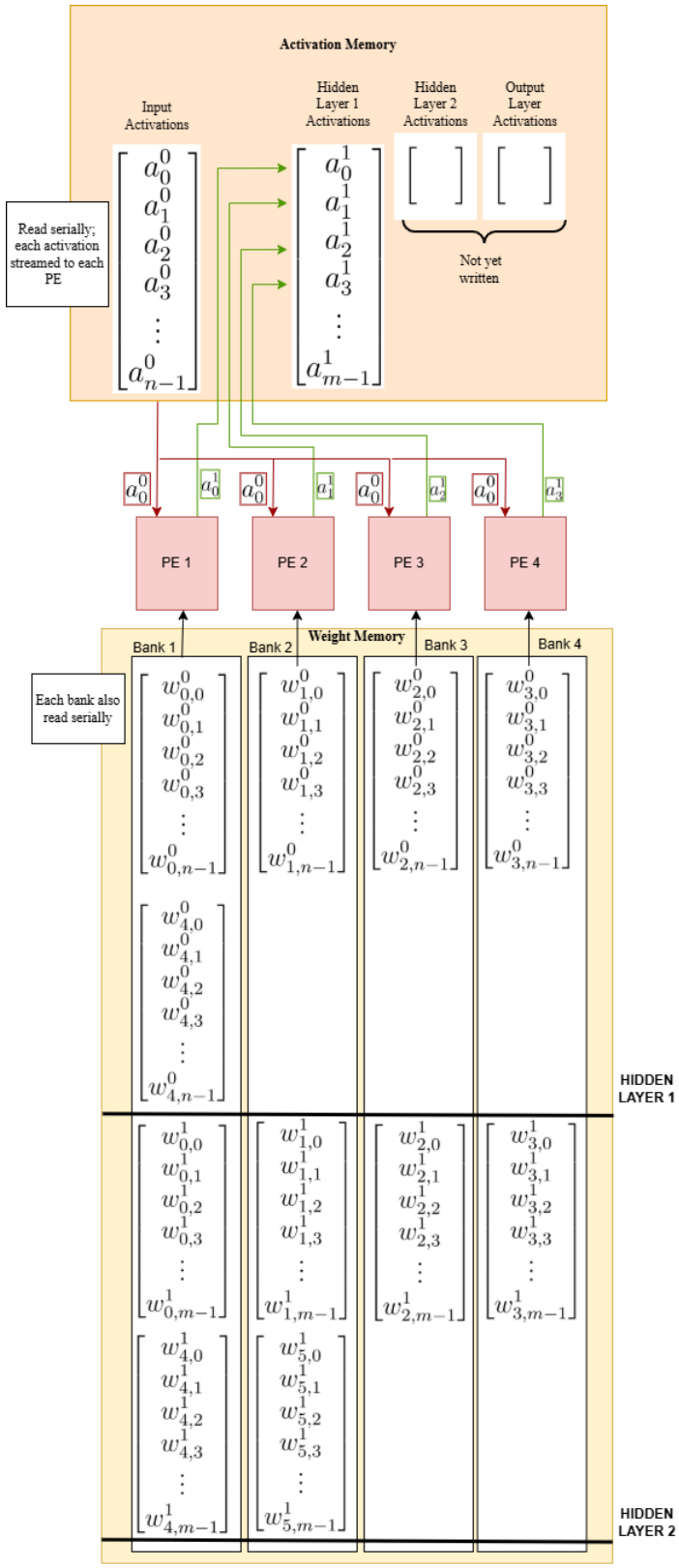

Figure 2: Memory interface with process engines.

Now, when it comes to actually transforming the weights into banked memory at compile time(?), we might be able to do the following: suppose all weights are stored contiguously. In other words, suppose the weight matrix in equation 1 is provided by David in the form of a 1D array W (in row-major order). To transform this into banked memory, then for each bank p, we store $W[p], W[p + NUM_PE], W[p + 2 \cdot NUM_PE], ...$ One important point is that the banks need to be read in parallel, so in terms of FPGA implementation, this will probably come down to physically separating the FPGA BRAMs.

It would also probably make sense to place the the corresponding bias at the end of each of the vectors shown in the weight memory, though it might also be perfectly fine to store the biases in a dedicated section of memory. I do not feel the need to say much about the biases, because implementing a bias memory block would be trivial compared to the weight memory. One other thing I want to note is that the vectors shown in Fig 2 wont actually be independent vectors in the weight memory; each bank would be a contiguous, flattened array of values. I chose to use one vector per tile per bank in the figure for the sake of readability, but again, independent vectors like that wont actually exist.